《数据结构与算法设计》
课程设计总结

学号：**2352018**

姓名：刘彦

专业：数据科学与大数据技术

2025 年 8 月

目　录

第一部分 算法实现设计说明.....1
1.1 题目.....1
1.2 软件功能.....1
1.3 设计思想.....3
1.4 逻辑结构与物理结构.....4
1.5 开发平台.....7
1.6 系统的运行结果分析说明.....7
1.7 操作说明.....9
第二部分 综合应用设计说明.....10
2.1 题目.....10
2.2 软件功能.....10
2.3 设计思想.....13
2.4 逻辑结构与物理结构.....16
2.5 开发平台.....17
2.6 系统的运行结果分析说明.....17
2.7 操作说明.....20
第三部分 实践总结.....22
3.1.所做的工作.....22
3.2.总结与收获.....22
第四部分 参考文献.....23

第一部分 算法实现设计说明

1.1 题目

堆的建立和筛选

输入一组关键值，用堆排序的方法进行从小到大的排序。

要求：(1)可以实现从小到大的排序，输出并显示该结果；

(2)随时显示输出堆顶元素，进行重新筛选后的堆。

1.2 软件功能

1.2.1 输入与控制模块

该模块负责接收用户输入和控制排序的整个流程。

A.输入功能

用户输入一个整数序列，表示为数组$A = [a_1, a_2, \dots, a_n]$。程序通过解析字符串，构建这个列表。实现方式是使用 QLineEdit 组件。start_button 的点击事件连接到 start_sort 方法，该方法负责解析文本框中的字符串，并将其转换为一个整数列表。

B.排序控制功能

软件提供了两个独立的按钮："交换堆顶与末尾元素"和"重新筛选"，让用户手动控制堆排序的每一步。核心操作是基于数组索引的数学计算：

- **构建大顶堆**：对数组A的$i = \left\lfloor \frac{n}{2} \right\rfloor - 1$到 0 的所有元素，执行$Heapify(A, n, i)$操作。
- **交换操作**：在每轮排序中，执行$A[0] \leftrightarrow A[n-1]$的交换，然后将堆的大小减 1，即$n \leftarrow n - 1$。
- **重新筛选**：在交换后，对新的根节点执行$Heapify(A, n-1, 0)$操作。

实现方式是使用两个 QPushButton 组件：swap_button 和 heapify_button。这两个按钮的状态（启用或禁用）会根据当前的排序步骤动态更新。例如，在"交换"操作完成后，会禁用"交换"按钮并启用"重新筛选"按钮，强制用户按照正确的步骤进行操作。start_button 用于初始化排序，一旦开始，该按钮会被禁用，直到排序完成。按钮的 clicked 信号分别连接到 swap_elements 和 reheapify 方法，这些方法会调用核心排序逻辑（HeapVisualizer 类）中的相应方法。

1.2.2 数据可视化模块

该模块负责将堆排序的动态过程以图形化方式呈现。

A.堆结构可视化功能

输入的数组以二叉树的形式展示，直观地表现堆的结构。程序将数组A视为一个**完全二叉树**。对于数组中索引为 i 的节点：

- 其父节点的索引为$\lfloor 2i-1 \rfloor$（如果$i>0$）。
- 其左子节点的索引为$2i+1$（如果$2i+1<n$）。
- 其右子节点的索引为$2i+2$（如果$2i+2<n$）。

节点位置计算：

- 节点 i 所在的层级为$L_i=\lfloor \log_2(i+1) \rfloor$。
- 节点 i 在其层级中的位置为$P_i=i-(2^{L_i}-1)$。
- 水平位置 x 和垂直位置 y 的计算：

$$x_i=\frac{ViewWidth}{2^{Li}+1}\times(P_i+1)+Padding$$

$$y_i=L_i\times LevelHeight+Padding$$

实现方式是使用 QGraphicsView 和 QGraphicsScene 作为绘图画布。自定义 HeapGraphicsView 类，该类继承自 QGraphicsView 并封装了绘图逻辑。update_heap 函数根据数组内容和当前堆的大小（heap_size）动态绘制节点和连接线。每个节点使用 addEllipse 绘制圆形，并使用 addText 显示节点值。节点的位置通过简单的数学计算确定，将二叉树的层次结构映射到二维平面上，确保不同层次的节点间有合适的间距。

B.状态高亮功能

区分堆中的元素和已排序的元素，并用不同颜色进行标记。

实现方式是 update_heap 函数根据索引 i 和 heap_size 的值来判断一个节点是属于堆的一部分还是已排序部分。对于数组中的任意元素$A[i]$，其颜色取决于它是否在当前堆中：

- 如果$i<heap_size$，该节点属于堆，用蓝色高亮。
- 如果$i\geq heap_size$，该节点已排序，用绿色高亮。

1.2.3 状态与日志模块

该模块提供文本信息反馈，帮助用户理解当前状态和排序过程。

A.实时状态显示功能

在界面上实时显示当前的操作状态、堆顶元素以及已排序的部分。

实现方式是使用 QLabel 组件来显示状态信息。update_status 函数根据 HeapVisualizer 的 current_step 属性更新 status_label 的文本。top_element_label 则实时显示堆的根节点值。

B.数组状态与结果显示功能

以文本形式展示排序过程中的数组变化，并最终显示完整的排序结果。

实现方式是使用两个 QLabel：current_array_label 和 final_result_label。current_array_label 在每次操作后（交换或重新筛选）更新为 heap_visualizer.arr 的当前状态。当排序完成后，final_result_label 会显示最终的完整排序结果。

C.操作日志功能

提供一个文本区域，记录每一步操作的详细信息，如“构建初始大顶堆”、“交换堆顶与末尾”等。

实现方式是使用 QTextEdit 组件作为日志输出区域。在每次操作相关的函数（如 start_sort、swap_elements、reheapify）中，使用 append 方法向 QTextEdit 添加新的日志条目，形成一个操作历史记录。

1.3 设计思想

本软件的设计核心思想是“交互式、模块化、可视化”，旨在让用户能直观、分步地理解堆排序算法的执行过程。传统的堆排序算法可视化工具通常是自动播放的，用户只能被动观看。而本软件通过赋予用户对每一步操作的控制权，将学习过程从被动的观察转变为主动的探索。

1.3.1 模块化设计

为了实现清晰、可维护的代码结构，我们将软件分为以下几个核心模块：

A.数据与算法模块（HeapVisualizer）：

专注于堆排序算法的核心逻辑，不涉及任何用户界面或绘图细节。创建一个 HeapVisualizer 类来封装堆排序的所有函数，例如 heapify（调整堆）、build_max_heap（构建大顶堆）和 swap_root_and_last（交换堆顶与末尾元素）。

这种分离使得算法逻辑可以独立测试和重用，任何对算法的修改都不会影响到用户界面，反之亦然。

B.可视化模块（HeapGraphicsView）：

专注于将数据和算法状态以图形化方式呈现。创建一个自定义的 QGraphicsView 子类，专门负责将数组数据渲染为二叉树结构。它通过 update_heap 函数接收数组和当前堆大小，然后计算每个节点的位置、绘制节点和连接线，并根据其状态（在堆中还是已排序）应用不同的颜色。将复杂的图形绘制逻辑封装在一个独立的组件中，使主窗口代码更简洁。

C.主界面与控制模块（App）

负责构建整个用户界面，连接各个模块，并处理用户交互事件。作为主应用程序窗口，它实例化 HeapVisualizer 和 HeapGraphicsView 对象。它监听按钮的点击事件，调用 HeapVisualizer 中的算法方法，然后调用 HeapGraphicsView 的更新方法来刷新显示。它还负责管理按钮的启用/禁用状态，以及在 QTextEdit 中记录操作日志。

它协调各个模块的工作，确保了流程的顺畅。

1.3.2 交互式流程设计

软件的交互式流程严格遵循堆排序算法的步骤，并通过按钮状态进行引导：

A.**初始化**：用户输入数据并点击“开始排序”。

- 后端：HeapVisualizer 接收数组$A = [a_1, a_2, \dots, a_n]$，并立即执行构建大顶堆的操作。该操作从数组的最后一个非叶子节点开始，向上遍历到根节点，对每个节点 i 调用$Heapify(A, n, i)$。
- 前端：界面更新，显示初始堆结构。“交换”按钮被启用，而“重新筛选”按钮被禁用，提示用户下一步操作。

B.**分步执行**

- 步骤一（交换）：用户点击“交换堆顶与末尾元素”。
  - 后端：HeapVisualizer 调用 swap_root_and_last，执行一次交换操作$A[0] \leftrightarrow A[n-1]$。然后将堆的大小减 1。
  - 前端：界面更新，显示交换后的数组和树结构。此时“交换”按钮被禁用，“重新筛选”按钮被启用。
- 步骤二（重新筛选）：用户点击“重新筛选”。
  - 后端：HeapVisualizer 调用 heapify_step，对新的堆顶元素执行$Heapify$操作，使其下沉到正确的位置，恢复大顶堆性质。这步操作的时间复杂度为$O(logn)$。
  - 前端：界面更新，显示筛选后的堆结构。此时“重新筛选”按钮被禁用，“交换”按钮被再次启用，进入下一轮循环。
- 排序完成
  - 判断：当堆的大小 n 减到 0 时，说明所有元素都已排序。
  - 结束：软件禁用所有控制按钮，并显示最终的排序结果，提示用户排序已完成。

1.4 逻辑结构与物理结构

1.4.1 逻辑结构

A.堆排序算法的逻辑结构

堆排序基于完全二叉树（Complete Binary Tree）的大顶堆性质，逻辑上通过数组索引模拟树结构。

● 完全二叉树索引关系

对于节点索引 i（从 0 开始）：左子节点：$left = 2i + 1$；右子节点：$right = 2i + 2$；父节点：$parent = \lfloor (i-1)/2 \rfloor$，这些公式用于定位子节点。

● 大顶堆性质

对于任意节点 i，其值满足：$A[i] \geq A[left]$且$A[i] \geq A[right]$，其中A是数组，算法保证确保子树 i 满足：

$$A[i] = \max(A[i], A[2i+1], A[2i+2]) \text{,若 } 2i+1 < n \text{且 } 2i+2 < n$$

执行逻辑是比较$A[i]$、$A[left]$、$A[right]$，将最大值的索引赋给$largest$，若$largest \neq i$，则交换$A[i]$和$A[largest]$，并递归调整子树。

● 构建初始大顶堆

算法从最后一个非叶子节点（索引$\lfloor n/2 \rfloor - 1$）开始，向根节点（索引 0）逆序调用 heapify：

$$\text{for } i = \lfloor n/2 \rfloor - 1 \text{ down to } 0, \text{call heapify}(n, i)$$

数学依据是在完全二叉树中，索引$\lfloor n/2 \rfloor - 1$是最后一个非叶子节点（因为节点$\lfloor n/2 \rfloor$到$n-1$是叶子节点）。时间复杂度$O(n)$，因为从下到上调整，每层节点数为2^k，调整高度为$h-k$，总工作量为：

$$\sum_{k=0}^{h-1} \frac{n}{2^{k+1}} \cdot (h-k) = O(n)$$

其中$h = \lfloor \log_2 n \rfloor$是树高。

● 堆排序主循环

每次迭代：交换堆顶（$A[0]$）与当前堆末尾（$A[n-1]$）：$A[0] \leftrightarrow A[n-1]$。堆大小 n 减 1，最大元素移到数组末尾（已排序部分）。对根节点调用 heapify(n,0)，调整剩余堆为大顶堆。

$$\text{for } n = \text{len}(A) \text{ down to } 1: A[0] \leftrightarrow A[n-1], n = n-1, \text{heapify}(n, 0)$$

每次 heapify 为$O(\log n)$，共 n 次迭代，总时间复杂度：$O(n \log n)$。

● 状态控制逻辑

算法跟踪状态（0：新排序，1：已建堆，2：已交换，3：已调整），对应于：

$$\text{state} \in \{0,1,2,3\}$$

堆大小 n 表示当前堆的节点数：

$$n \in [0, original_len]$$

已排序部分：$A[n: original_len]$，从大到小排列。

B.图形化可视化逻辑

图形化显示将逻辑上的二叉堆映射为二维平面坐标，计算节点位置和连接线。

● 节点层级计算

节点 i 所在层级为$level = \lfloor \log_2(i+1) \rfloor$，同一层级的节点数为$nodes_in_level = 2^{level}$，节点在层内的相对位置$pos_in_level = i - (2^{level} - 1)$。

● 坐标计算

水平间距：

$$h_{spacing} = \frac{view_width}{2^{level} + 1}$$

其中$view_width = width - 2 \cdot node_radius$。垂直间距为常数。

节点 i 的坐标为：

$$x = h_{spacing} \cdot (pos_in_level + 1) + node_radius$$

$$y = level \cdot level_height + node_radius$$

对于节点 i，连接到左子节点 2i+1 和右子节点 2i+2。

C.用户交互逻辑

按钮控制逻辑分为状态转移和按钮启用条件：

$$current_step: 0 \to 1\ (start_sort) \to 2\ (swap_elements) \to 3\ (reheapify) \to 2\ or\ end$$

$$swap_button.enabled = (current_step \in \{1,3\}) \wedge (n > 0)$$

$$heapify_button.enabled = (current_step = 2) \wedge (n > 0)$$

$$start_button.enabled = (n = 0) \vee (sort\ completed)$$

1.4.2 物理结构

● 数组存储

堆部分：$arr[0:n]$，已排序部分：$arr[n: original_len]$，物理上为连续内存块。

$$arr = [a_0, a_1, \dots, a_{original_len-1}], size = O(n)$$

● 图形存储

QGraphicsScene 存储椭圆、文本和线段，总存储：$O(n)$（每个节点一个椭圆+文本，边最多$n-1$）。

$$Items = \{Ellipse_i, Text_i \mid 0 \le i < original_len\} \cup \{Line_{i,j} \mid j = 2i+1, 2i+2\}$$

- 坐标存储

$$positions = \{(i, (x_i, y_i)) \mid 0 \le i < original_len\}, x_i, y_i \in R$$

1.5 开发平台

本软件采用 Python 3.10 作为主要开发语言，结合 PyQt6 模块实现图形用户界面（GUI）和相关功能。Python 3.10 在 Windows 11 上具有良好的兼容性，适合开发跨平台应用和数学计算任务。

①Python 标准库

- sys：Python 3.10 内置模块，用于系统级操作，例如获取命令行参数、操作系统信息或管理程序退出。无需额外安装，无任何第三方或开源代码依赖。

- math：Python 3.10 内置模块，提供标准数学函数（如三角函数、对数、平方根）和常量（如 π、e）。无需安装，直接可用，无外部依赖。

②第三方程序包

- PyQt6

基于 Qt 6 框架的 Python 绑定库，用于构建跨平台的 GUI 应用。在本项目中，PyQt6 用于创建交互界面，包括主窗口、按钮、文本框等控件，以及信号槽机制处理用户交互事件。

在代码中，PyQt6 提供了以下模块：

- PyQt6.QtWidgets：用于界面组件，如 QApplication（应用程序实例）、QWidget（主窗口）、QVBoxLayout 和 QHBoxLayout（布局管理）、QLineEdit（输入框）、QPushButton（按钮）、QTextEdit（日志输出）、QLabel（标签）、QGraphicsView 和 QGraphicsScene（图形视图）、QFrame（分组框架）。

- PyQt6.QtGui：用于图形绘制和样式，包括 QPen（画笔）、QBrush（填充）、QColor（颜色）、QFont（字体）。

- PyQt6.QtCore：提供核心功能，如 Qt 枚举（用于设置颜色、对齐方式等）。

代码中使用 PyQt6 实现交互式界面和堆的树形可视化（通过 QGraphicsView 和 QGraphicsScene 绘制节点和连接线）。PyQt6 版本为 6.9.1，与 Python 3.10.23 兼容，代码未使用 PyQt6 的扩展模块（如 PyQt6-Charts 或 PyQt6-WebEngine），保持依赖最小化。

1.6 系统的运行结果分析说明

首先我先完成了堆排序算法逻辑的代码调试，在命令行中模拟出堆的结构和元素的交换过程，并以数组$[4, 1, 7, 3, 8, 5, 2, 6, 9, 0, 10, 12, 11]$检验，检查每一步操作后数组的变化是否符

合预期：

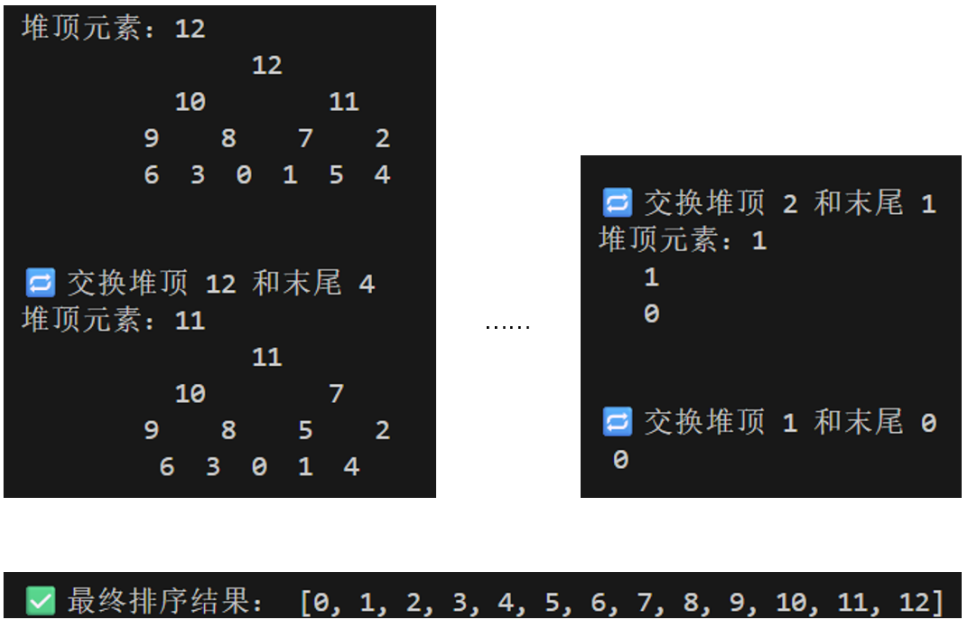

图 1 堆排序算法调式结果

然后使用 PyQt6 提供的工具进行交互界面的制作，调试界面主要关注控件的摆放、大小策略以及在窗口尺寸变化时的自适应行为。

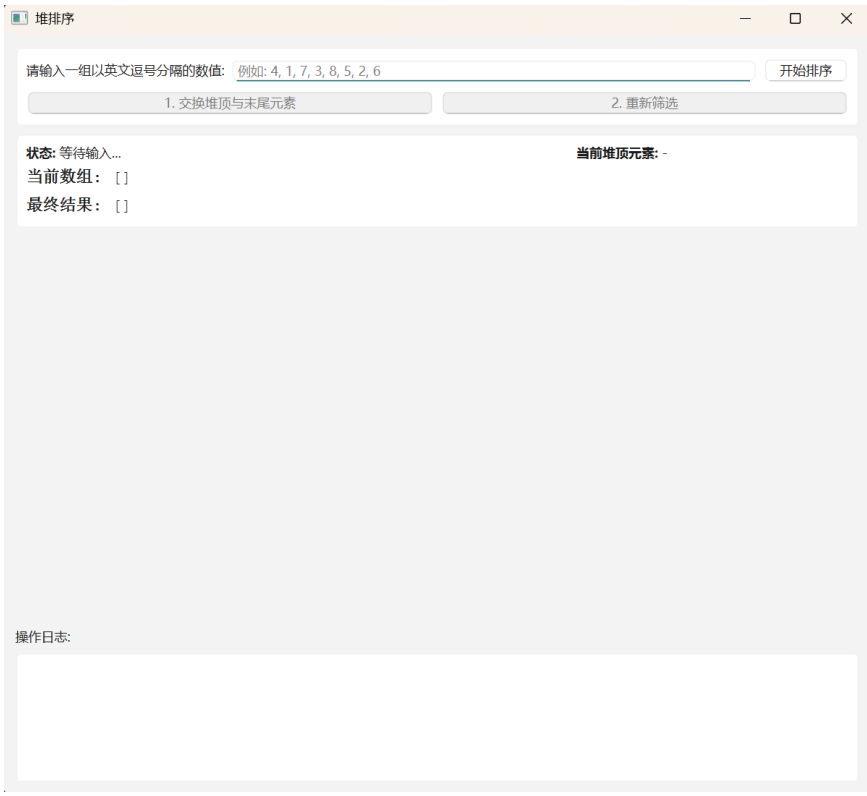

图 2 堆排序软件交互界面模型图

算法能够正确地将一个无序数组构建成一个大顶堆，然后通过循环地将堆顶元素（当前

最大值）移动到数组末尾并调整剩余元素，最终得到一个升序排列的数组。在排序过程中，通过观察已排序部分判断算法的正确性：

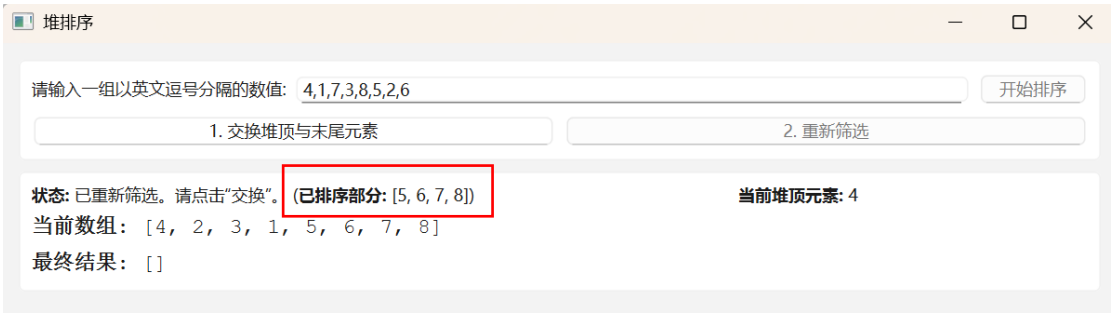

图 3 排序过程示意图

需要明确的是，堆排序本身是一种不稳定的排序算法。这意味着如果数组中存在两个相等的元素，排序后它们的相对位置可能会改变。但软件在没有用户操作时处于稳定的等待状态，它能可靠地响应用户的按钮点击和窗口操作。

如果用户输入非数字字符（如字母、特殊符号），程序不会崩溃，而是会在日志区提示“输入格式错误...”。如果用户输入为空或只包含逗号，程序同样会提示“请输入有效的数值。”，避免了因空数组导致的潜在错误。软件通过禁用/启用控制按钮（“交换”和“重新筛选”），软件强制用户按照“交换、筛选、交换……”的正确流程进行操作，防止了因误操作导致的逻辑混乱。例如，在完成“交换”之前，用户无法点击“重新筛选”。

图 4 堆排序软件输入错误处理示意图

1.7 操作说明

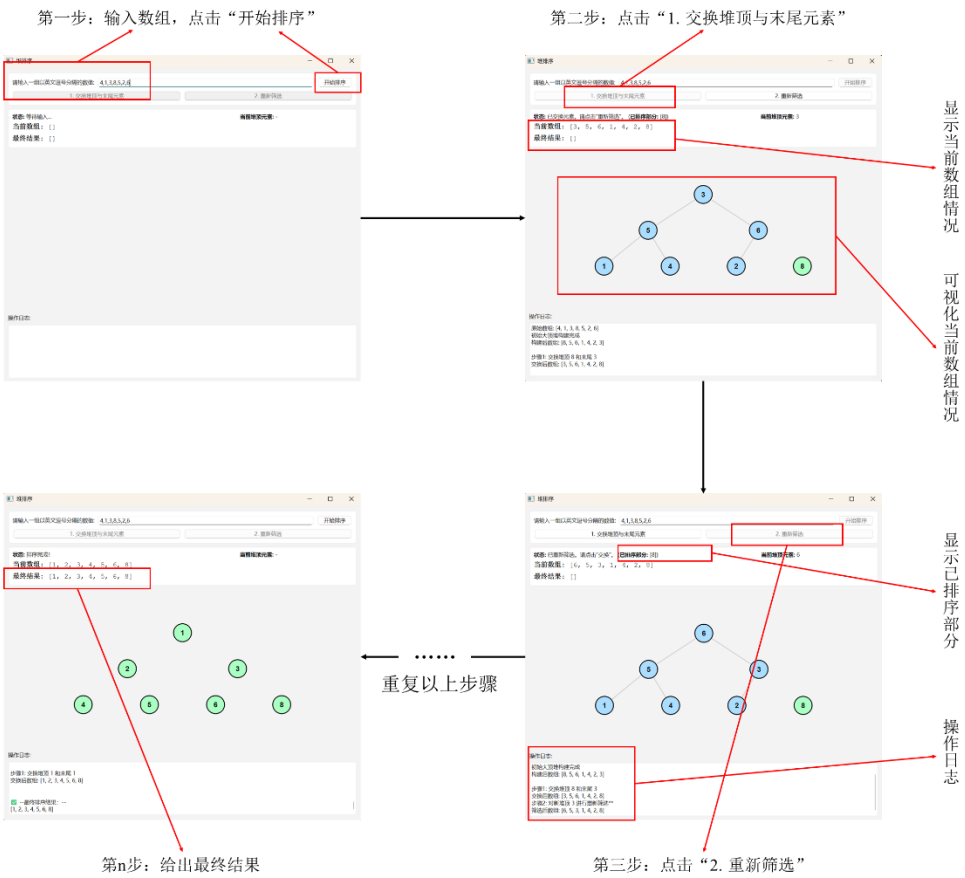

图 5 堆排序软件操作说明图

第二部分 综合应用设计说明

2.1 题目

现需开发一个简单的文本编辑器（支持 200 字以内的输入即可），能支持输入数据的形式和范围包括大写、小写的英文字母、任何数字及标点符号。该编辑器需要实现以下功能：

(1)对输入的文字进行统计，统计出文字、数字、空格的个数；

(2)统计某一字符串在文章中出现的次数，并输出该次数；

(3)删除某一子串，并将后面的字符前移；

(4)保存文本修改内容，可以撤销修改和恢复修改。

2.2 软件功能

本软件是一个基于 PyQt6 框架开发的简单文本编辑器，支持字符输入限制、实时统计、子串操作、撤销/恢复、文件保存，以及文本与 16 进制编码的相互转换。软件界面包括主窗

口、菜单栏、工具栏和状态栏，核心组件为 QTextEdit 文本编辑区。

2.2.1 实时字符统计功能

软件实时统计并显示文本中的中文字符数、英文字母数、数字数、英文单词数、空格数、换行符数、制表符数，以及总字符数。统计区分中文（Unicode 范围 \u4e00-\u9fff）和英文（a-z/A-Z），英文单词基于空格分割并检查是否包含字母。统计结果在状态栏实时更新，支持多行文本。

实现方式是通过 QTextEdit 的 textChanged 信号连接到 on_text_changed，进而调用 update_status 更新状态栏。

逻辑流程：

- **获取文本**：text = self.text_edit.toPlainText()。
- **中文统计**：sum(1 for char in text if '\u4e00' <= char <= '\u9fff')。
- **英文字母统计**：sum(1 for char in text if 'a' <= char.lower() <= 'z')（忽略大小写）。
- **数字统计**：sum(1 for char in text if char.isdigit())。
- **英文单词统计**：先 text.split() 分割潜在单词，然后 sum(1 for word in potential_words if any('a' <= char.lower() <= 'z' for char in word)) 检查是否含英文字符。
- **空格/换行/制表符**：使用字符串方法 text.count(" ")、text.count("\n")、text.count("\t")。
- **总字符**：len(text)。

2.2.2 子串出现次数统计功能

用户可输入指定子串，软件统计其在当前文本中出现的次数（区分大小写，支持重叠或非重叠）。若文本为空，提示用户；结果通过消息框显示。仅在文本模式下可用。

实现方式：

- 获取文本 text = self.text_edit.toPlainText()，若为空，提示“编辑器内容为空”。
- 使用 QInputDialog.getText 获取子串 substring。
- 计算次数 count = text.count(substring)（支持非重叠计数）。

2.2.3 子串删除功能（三种模式）

提供三种删除指定子串的模式：

(3a)删除全部匹配子串：移除所有出现的子串。

(3b)删除第一个匹配子串：仅移除首次出现的子串。

(3c)删除第 N 个匹配子串：用户指定 N（1 到总次数），移除第 N 次出现的子串。

所有模式需输入子串，若未找到则警告；操作后更新文本并提示成功。

实现方式是利用编辑菜单下的“删除子串”子菜单，分别触发 delete_all_substrings、delete_first_substring、delete_nth_substring。

所有模式的通用逻辑：

- 获取文本 text = self.text_edit.toPlainText()。
- 使用 QInputDialog.getText 获取子串 substring。
- 检查是否存在 if substring not in text，若无，警告“文中未找到字符串'xxx'”。

(3a)删除全部：使用 new_text = text.replace(substring, '') （替换所有），设置 self.text_edit.setPlainText(new_text)，提示“已删除所有出现的'xxx'”。

(3b)删除第一个：使用 new_text = text.replace(substring, '', 1)（限替换 1 次），其余同上，提示“已删除第一个出现的'xxx'”。

(3c)删除第 N 个：先计算总次数 total_count = text.count(substring)，若 0 则警告；使用 QInputDialog.getInt 获取 N（范围 1-total_count，默认 1）；然后循环查找位置：for i in range(n): start_index = text.find(substring, start_index + 1)（从上个位置后搜索）；最后构建新文本 new_text = text[:start_index] + text[start_index + len(substring):]，设置并提示“已删除第 N 个出现的'xxx'”。

2.2.4 撤销和恢复功能

支持标准的撤销（Undo）和恢复（Redo）操作，与主流编辑器（如 Word）逻辑一致：撤销上一步操作，恢复撤销的步骤。快捷键 Ctrl+Z（撤销）、Ctrl+Shift+Z（恢复）。菜单和工具栏提供图标支持。

实现方式是利用 QTextEdit 内置的 undo()和 redo()方法，直接连接到动作 self.undo_action.triggered.connect(self.text_edit.undo) 和 self.redo_action.triggered.connect(self.text_edit.redo)。QTextEdit 内部维护撤销栈（基于 QUndoStack），自动记录插入/删除/替换等操作。用户触发时，直接调用内置方法，无需自定义栈。修改完成后还可以将当前文本保存为 UTF-8 编码的.txt 文件。

2.2.5 文本与 16 进制编码相互转换功能

在原窗口支持文本（UTF-8）和 16 进制编码的切换：

- 文本⇒hex：将文本编码为 UTF-8 字节，每字节转为两位小写 hex（空格分隔）。
- hex⇒文本：解析 hex（忽略空格/换行，支持自由格式），解码为 UTF-8 文本。

在 hex 模式下，禁用字符限制和统计，允许直接编辑 hex 内容（如修改字节）。切换后更新菜单文本/图标/提示。仅转换有效内容，空文本提示。

逻辑流程：

- 文本⇒hex：

● 获取文本 text = self.text_edit.toPlainText()，若空提示。 ● 编码 text.encode('utf-8')，生成 hex ' '.join(f'{b:02x}' for b in bytes)。 ● 设置文本 self.text_edit.setPlainText(hex_representation)，self.is_hex_mode = True。

- hex⇒文本

● 获取 hex 文本 hex_text = self.text_edit.toPlainText()，若空提示。 ● 清理' '.join(hex_text.split()) 移除空格/换行。 ● 若非空：bytes.fromhex(cleaned_hex).decode('utf-8') 解码。 ● 设置文本，self.is_hex_mode = False。

2.3 设计思想

2.3.1 实现思路

软件采用分层架构与 FSM 模型：

● 分层架构：分为 UI 层（QMainWindow、QTextEdit、QMenuBar）、逻辑层（槽函数处理信号，如 textChanged→on_text_changed）和数据层（字符串/字节操作）。信号-槽机制确保实时响应，例如文本变化触发统计更新。

● 状态管理：定义状态集$\sigma \in \{0{:}文本, 1{:}16\ 进制\}$，通过布尔变量 is_hex_mode 管理，转移函数$f(\sigma,\text{event}) \to \sigma'$由 toggle_hex_conversion 实现（如 Ctrl+H 触发$\sigma \to 1-\sigma$）。此设计隔离模式功能：文本模式启用限制与统计，16 进制模式支持自由编辑。

● 优化与容错：使用 blockSignals 防止递归信号，try-except 捕获异常（如 ValueError），通过 QMessageBox 反馈。性能优化仅在必要时更新（如状态栏）。总响应时间$T = O(n) + c$，其中$n \leq 200$，c 为 UI 常数开销。

2.3.2 数据结构选择

数据结构选择基于问题规模（文本长度$m \leq 200$）和 Python 优化，优先 immutable 类型

（如 str、bytes）确保数据安全，mutable 类型（如 list）用于临时计算。设计思想为将文本抽象为线性序列或子集，映射到数学模型，便于算法应用。

● 字符串（str）

文本编辑核心为 Unicode 序列，str 是 immutable，支持$O(1)$索引和$O(n)$操作（如 count/replace），内存紧凑（CPython 以 char*存储）。相较 list，str 更适合频繁访问。

设计思想为抽象为序列$S=(s_1,s_2,\dots,s_m)$，$s_i\in$ Unicode，immutable 利于撤销栈（QUndoStack 存储差异$\Delta S=S'-S$）。16 进制模式下，S 表示格式化 hex 字符串，解析为集合$\{h_i\}$，其中$h_i=hex(b_i)$：

$$|S|=\sum_{i=1}^{m}1\,,Q\subseteq S\Leftrightarrow\exists i{:}\,S[i{:}\,i+|Q|]=Q$$

● 字节序列（bytes）

16 进制转换需字节级操作，bytes 是 immutable 字节数组，支持 encode/decode 和 fromhex，效率为$O(k)$（k 字节）。

设计思想为映射为字节流$B=(b_1,b_2,\dots,b_k)$，$b_i\in[0,255]$，定义双射$B=encode(S,utf-8)$，$S=decode(B)$：

$$|B|=\sum_{s_i\in S}byte_len\big(\text{utf-8}(s_i)\big)$$

● 列表（list）

用于临时拆分（如 split()生成单词列表），支持$O(n)$迭代，mutable 适合动态过滤。规模小（单词数$p\ll n$）时高效。

设计思想是抽象为集合$W=\{w_1,w_2,\dots,w_p\}$，过滤英文单词子集$W_{en}=\{w\in W\mid\exists c\in w{:}\ll CODE_BLOCK_10\gg z'\}$：

$$|W_{en}|=\sum_{w\in split(S)}I(\exists c\in w{:}\ll CODE_BLOCK_11\gg z')$$

其中 I 为指示函数：$I(true)=1$，$I(false)=0$。

● 布尔（bool）和整数（int）

is_hex_mode 和 MAX_CHARS 是轻量标志，无内存开销，适合状态控制和阈值设置。设计思想是利用 FSM 模型，状态转移$\sigma'=\sigma\oplus 1$：

$$\sigma\in\{0,1\},MAX_CHARS=200$$

2.3.3 算法设计

算法以$O(n)$或$O(kn)$复杂度为主，确保实时性。

● 字符输入限制算法

获取 S；2 若$|S| > 200 \wedge \sigma = 0$，则$S' = S[1{:}201]$；然后设置光标，更新视图。

$$|S'| = \min(|S|, 200), S' = S[1{:}\min(|S|, 200)]$$

● 实时字符统计算法

获取 S；单遍扫描计算计数；更新状态栏。

$$C_{ch} = \sum_{i=1}^{m} I(\ll CODE_BLOCK_14 \gg \backslash u9fff')$$

$$C_{en} = \sum_{i=1}^{m} I(\ll CODE_BLOCK_15 \gg z')$$

$$C_{word} = \sum_{w \in split(S)} I(\exists c \in w{:} \ll CODE_BLOCK_16 \gg z')$$

$$C_{sp} = \sum_{i=1}^{m} I(s_i = \ll CODE_BLOCK_17 \gg)$$

● 子串出现次数统计算法

输入 Q；计算$k = \mathrm{count}(S, Q)$。

$$k = |\{i \mid 1 \le i \le m - |Q| + 1, S[i{:}\, i + |Q|] = Q\}|$$

● 子串删除算法

输入 Q，校验$Q \in S$；计算位置集 P；根据模式重构 S'。

删除全部：

$$S' = \mathrm{replace}(S, Q, ``), \ |S'| = |S| - k \cdot |Q|$$

删除第一个：

$$p_1 = \min\{\, i \mid S[i{:}\, i + |Q|] = Q\}, S' = S[1{:}\, p_1] \cdot S[p_1 + |Q|{:}\, m]$$

删除第 N 个：

$$P = \{p_j \mid p_1 = \mathrm{find}(Q, 1), p_j = \mathrm{find}\big(Q, p_{j-1} + 1\big)\}, S' = S[1{:}\, p_N] \cdot S[p_N + |Q|{:}\, m]$$

● 撤销和恢复算法

栈$\Sigma = [\Delta_1, \Delta_2, \dots, \Delta_t]$，undo：$S' = S - \Delta_t$。

$$S' = S \ominus \mathrm{pop}(\Sigma)$$

● 文本与 16 进制转换算法

文本→hex：$B = \mathrm{encode}(S)$，$H = \mathrm{join}(\{\mathrm{hex}(b_i)\})$；hex→文本：$H' = \mathrm{join}\big(\mathrm{split}(H)\big)$，$B = \mathrm{fromhex}(H')$，$S = \mathrm{decode}(B)$。

$$\mathrm{hex}(b_i) = \left\lfloor \frac{b_i}{16} \right\rfloor \cdot 16^1 + (b_i \bmod 16) \cdot 16^0, b_i = \sum_{j=0}^{1} d_j \cdot 16^{1-j}$$

2.4 逻辑结构与物理结构

2.4.1 逻辑结构

该软件核心为线性结构（有序序列），辅助集合结构（无序分组）和状态结构（FSM 转移）。这允许将文本问题分解为可计算组件，如子串匹配视为序列子集。

● 线性结构

文本数据逻辑上为有序序列，支持插入、删除和遍历。字符串 S 表示为链式元素，支持子串操作。字节序列 B 类似，用于转换。

$$S = (s_1, s_2, \dots, s_m), m = |S| = \sum_{i=1}^{m} 1, Q = S[i{:}\, i + |Q|] \text{ for some } i$$

线性结构匹配文本的自然顺序，便于算法如 find（迭代位置 p）。

● 集合结构

统计类别（如中文字符）逻辑为无序集合，单词拆分为基于分隔符的集合。hex 模式下，字节视为有序对集合$\{(i, b_i)\}$。

$$C_{\mathrm{ch}} = \{s_i \in S \mid \ll\text{CODE_BLOCK_0}\gg \backslash u9fff'\}, |C_{\mathrm{ch}}| = \sum I\big(\mathrm{predicate}(s_i)\big)$$

$$W = \mathrm{split}(S) = \{w_1, w_2, \dots, w_p\}, W_{\mathrm{en}} \subseteq W$$

集合抽象忽略顺序，聚焦成员资格，便于过滤和计数。

● 状态结构

FSM 逻辑，状态 σ 与事件驱动转移，支持模式隔离。

$$\sigma \in \{0,1\}, \sigma' = f(\sigma, event) = \sigma \oplus 1\ (for\ toggle)$$

简化条件逻辑，16 进制状态转移矩阵确保一致性。

2.4.2 物理结构

物理结构基于 CPython 的内存布局，强调连续存储和指针引用，支持快速访问。字符串/字节用数组实现，list 用动态数组，标志用基本类型。

● 数组式结构

字符串 str 在内存中为连续 Unicode 数组（PyUnicodeObject，内部 char*或 wchar_t[]），支持$O(1)$随机访问。bytes 为 uint8_t[]字节数组。list 为 PyListObject 的动态数组（指针数组+容量扩展）。

$$\text{访问时间} = O(1), \quad \text{内存} = O(m) \text{ for } S, \quad \text{扩展} = 2 \times \text{capacity (for list)}$$

连续存储优化缓存命中，适合线性扫描算法。

● 标志结构

bool/int 为单一机器字（1/4/8 字节），栈或堆分配。最小开销，支持快速状态检查。

$$\text{内存}=O(1), \quad \sigma = \begin{cases} 0 & \text{文本} \\ 1 & \text{16 进制} \end{cases}$$

先识别核心数据（S、B、W、σ），抽象逻辑结构（序列、集合、FSM），映射物理（数组、变量），结合公式验证效率（如$O(1)$访问确保实时性）。

2.5 开发平台

本软件采用 Python 3.10 作为主要开发语言，结合 PyQt6 模块实现图形用户界面（GUI）和相关功能。Python 3.10 在 Windows 11 上具有良好的兼容性，适合开发跨平台应用和数学计算任务。

①Python 标准库

● sys：Python 3.10 内置模块，用于系统级操作，例如获取命令行参数、操作系统信息或管理程序退出。无需额外安装，无任何第三方或开源代码依赖。

②第三方程序包

● PyQt6

基于 Qt 6 框架的 Python 绑定库，用于构建跨平台的 GUI 应用。在本项目中，PyQt6 用于创建交互界面，包括主窗口、按钮、文本框等控件，以及信号槽机制处理用户交互事件。

在代码中，PyQt6 提供了以下模块：

- PyQt6.QtWidgets：用于界面组件，如 QApplication（应用程序实例）、QWidget（主窗口）、QVBoxLayout 和 QHBoxLayout（布局管理）、QLineEdit（输入框）、QPushButton（按钮）、QTextEdit（日志输出）、QLabel（标签）、QGraphicsView 和 QGraphicsScene（图形视图）、QFrame（分组框架）。
- PyQt6.QtGui：用于图形绘制和样式，包括 QPen（画笔）、QBrush（填充）、QColor（颜色）、QFont（字体）。

2.6 系统的运行结果分析说明

首先我先在命令行中实现了了文本编辑器的基本逻辑的代码调试，模拟出输入、统计、查询、删除和撤销的过程：

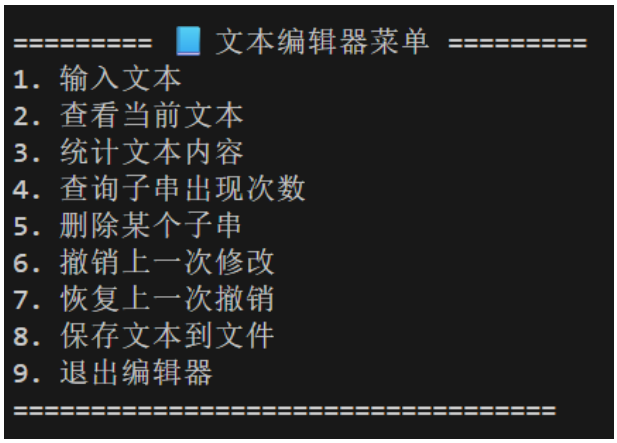

图 6 命令行文本编辑器示意图

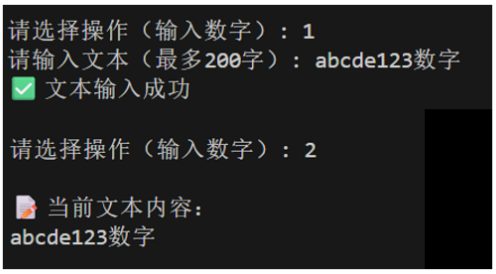

图 7 命令行文本编辑器操作示意图

然后使用 PyQt6 提供的工具进行交互界面的制作，调试界面主要关注控件的摆放、大小策略以及在窗口尺寸变化时的自适应行为。

图 8 文本编辑器交互界面模型图

图 9 文本编辑器字符数统计功能示意图

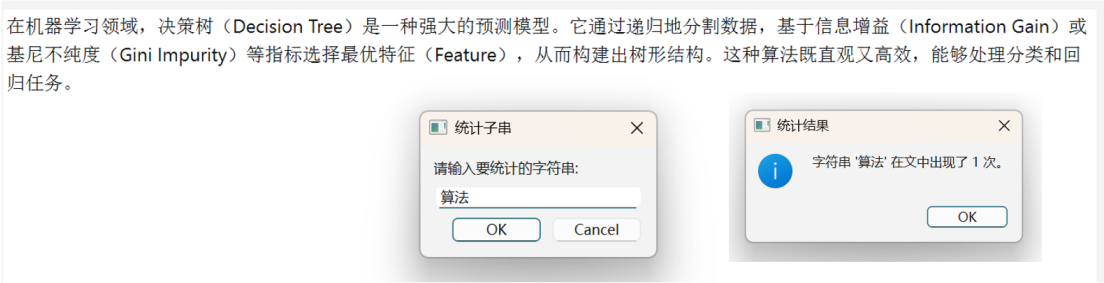

图 10 文本编辑器字符串统计功能示意图

图 11 文本编辑器删除子串功能示意图

软件所有核心功能，包括字符数限制、实时分类统计、子串的查找与三种模式删除、撤

销恢复、文件保存以及新增的文本与 16 进制相互转换，经案例测试证明，其运行结果均与设计预期完全一致。

软件基于 PyQt6 框架，事件循环和对象管理稳定可靠，且对字符数上限进行了有效控制，防止了无限制输入可能导致的性能问题。在程序化修改文本框内容时（如超限截断），使用了 blockSignals(True)和 blockSignals(False)，防止了不必要的递归或连锁反应，是保证系统稳定性的关键。

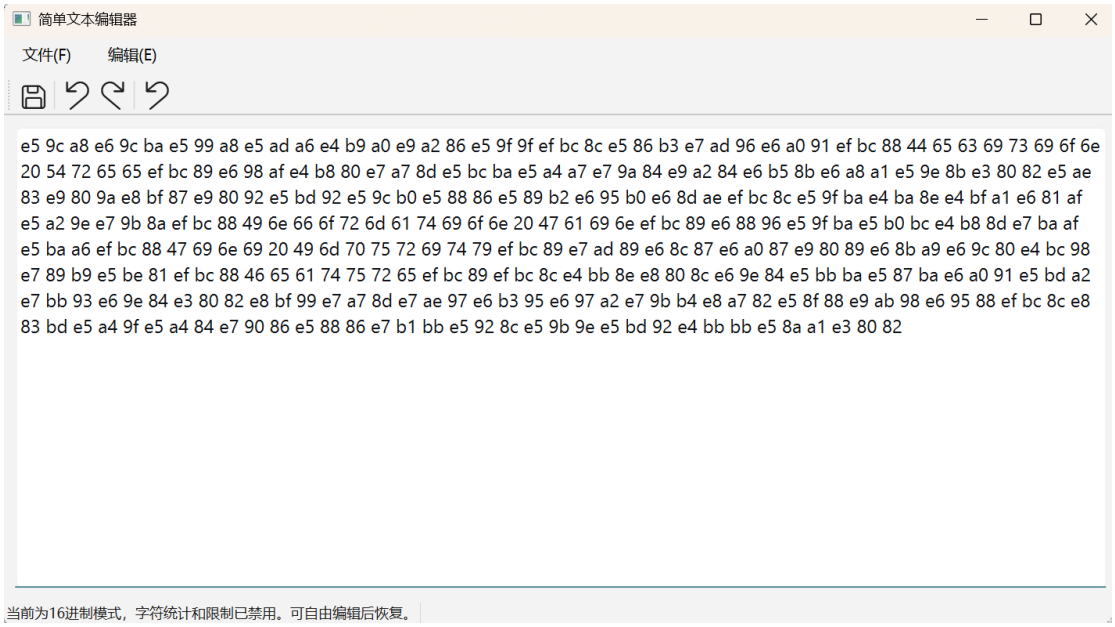

图 11 文本编辑器转换 16 进制功能示意图

软件对用户的输入进行了有效性检查，例如，在删除不存在的子串或处理空文本时，会给出清晰的提示信息，而不是抛出异常。在关键的、可能发生运行时错误的操作中（如文件保存、16 进制转文本），都使用了 try...except 结构进行异常捕获。这使得即使用户输入了格式错误的 16 进制码或选择了没有写入权限的路径，程序也不会崩溃，而是会向用户弹出明确的错误提示对话框。

2.7 操作说明

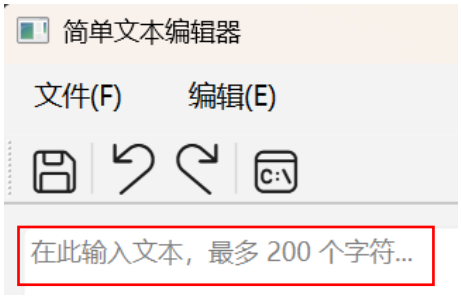

图 12 文本编辑器操作说明图 A

软件运行后，直接在文本框中输入文本。

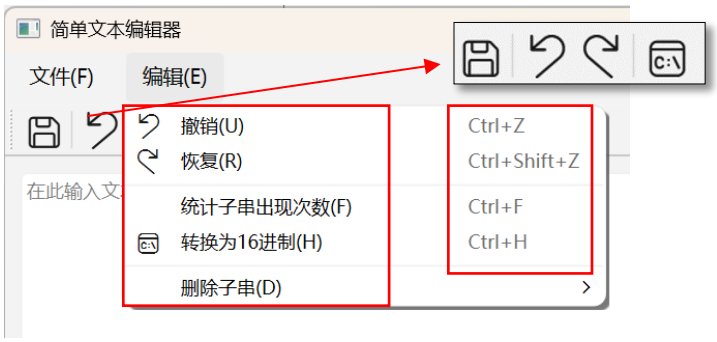

图 13 文本编辑器操作说明图 B

需要操作时，点击“编辑”按钮，可以进行撤销、恢复、统计子串出现次数、转换为 16 进制和删除子串操作，每个操作都有相应快捷键，撤销、恢复和转换为 16 进制操作还有便捷按钮。

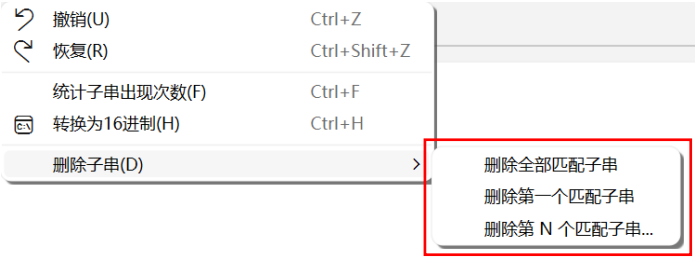

图 14 文本编辑器操作说明图 C

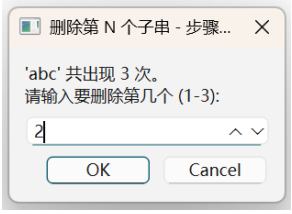

图 15 文本编辑器操作说明图 D

删除操作共 3 种，前两种直接输入要删除的子串，若要删除第 N 个出现的子串，还要指出 N 的值。

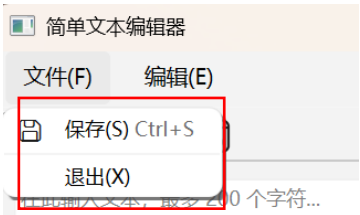

图 16 文本编辑器操作说明图 E

需要保存时，点击“保存”按钮，可以进行文件保存。

第三部分 实践总结

3.1.所做的工作

本次课程设计中，我成功设计并开发了两个功能完整的桌面应用程序，全面锻炼了从算法实现到用户界面开发的全过程能力。具体完成的工作内容总结如下：

①完成了“交互式堆排序可视化工具”，我使用 Python PyQt6 框架成功开发了一款软件，它通过将抽象的数组操作动态渲染为直观的二叉树图形，来帮助用户深入理解堆排序算法。该工具的核心特色在于其交互性，用户可以通过分步控制按钮手动执行“交换”与“重新筛选”等关键操作，并通过颜色高亮、实时状态更新和详细日志等多维度反馈来观察算法的每一步变化。为了确保软件的健壮性和可用性，我采用了算法与界面分离的模块化设计，并加入了周全的输入验证与错误处理机制。

②完成了“多功能简单文本编辑器”，我同样基于 PyQt6 构建了一个功能丰富的文本处理工具，它在实现标准编辑、撤销恢复及文件保存等基础功能之上，集成了一系列高级实用模块。该编辑器能够实时对输入内容进行多维度统计（如中文字数、单词数等），并提供了强大的子串操作功能，包括精确查找和三种不同的删除模式。其最具创新性的设计是内置了一个支持双向转换和在线编辑的文本与 16 进制编码器，极大地便利了数据分析。整个软件在开发过程中贯穿了容错设计思想，通过全面的异常捕获和清晰的用户提示，确保了程序的高度稳定性和易用性。

3.2.总结与收获

本次课程设计极大地提升了我的自学能力与实践动手能力，它驱动我从零开始掌握了 PyQt6 这一 GUI 框架，并将课堂上学到的数据结构与算法知识真正应用到了实际问题的解决中。通过将堆排序算法的每一步操作进行可视化，以及为文本编辑器实现复杂的数据统计与转换功能，我深刻体会到了理论与实践相结合的重要性，不仅巩固了对核心概念的理解，也锻炼了独立查阅文档、调试代码和攻克技术难题的综合能力。

更重要的是，这次实践培养了我初步的软件工程思维和用户导向的设计意识。在开发过程中，两个项目都遵循了模块化的设计思想。例如，将堆排序的算法逻辑与图形绘制分离，将文本编辑器的 UI 与功能函数分离。这种设计不仅使代码结构更清晰、易于维护，也让开发和调试过程更加高效。在开发过程中，我开始自觉地从用户的角度思考问题。如何让界面布局更合理？如何提供清晰的操作指引？如何处理用户的错误输入？这些思考让我明白，一

个好的软件不仅要功能正确，更要稳定、易用、容错性强。交互式按钮的引导和友好的错误提示框正是这一思想的体现。

第四部分 参考文献

[1] Riverbank Computing, PyQt6 Reference Guide. Riverbank Computing, 2024. Retrieved from https://www.riverbankcomputing.com/static/Docs/PyQt6/

[2] Gamma E., Helm R., Johnson R. & Vlissides J., Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional, 1994.

[3] Cormen T. H., Leiserson C. E., Rivest R. L. & Stein C., Introduction to Algorithms (4th ed.). MIT Press, 2022.

[4] Yergeau F., RFC 3629: UTF-8, a transformation format of ISO 10646. Internet Engineering Task Force (IETF), 2003.